第5章 单元测试和集成测试

《软件测试与质量保证》

马樱

Tel: 18359703871

y.mai@hit.edu.cn

计算学部 软件学院

5.1 什么是单元测试

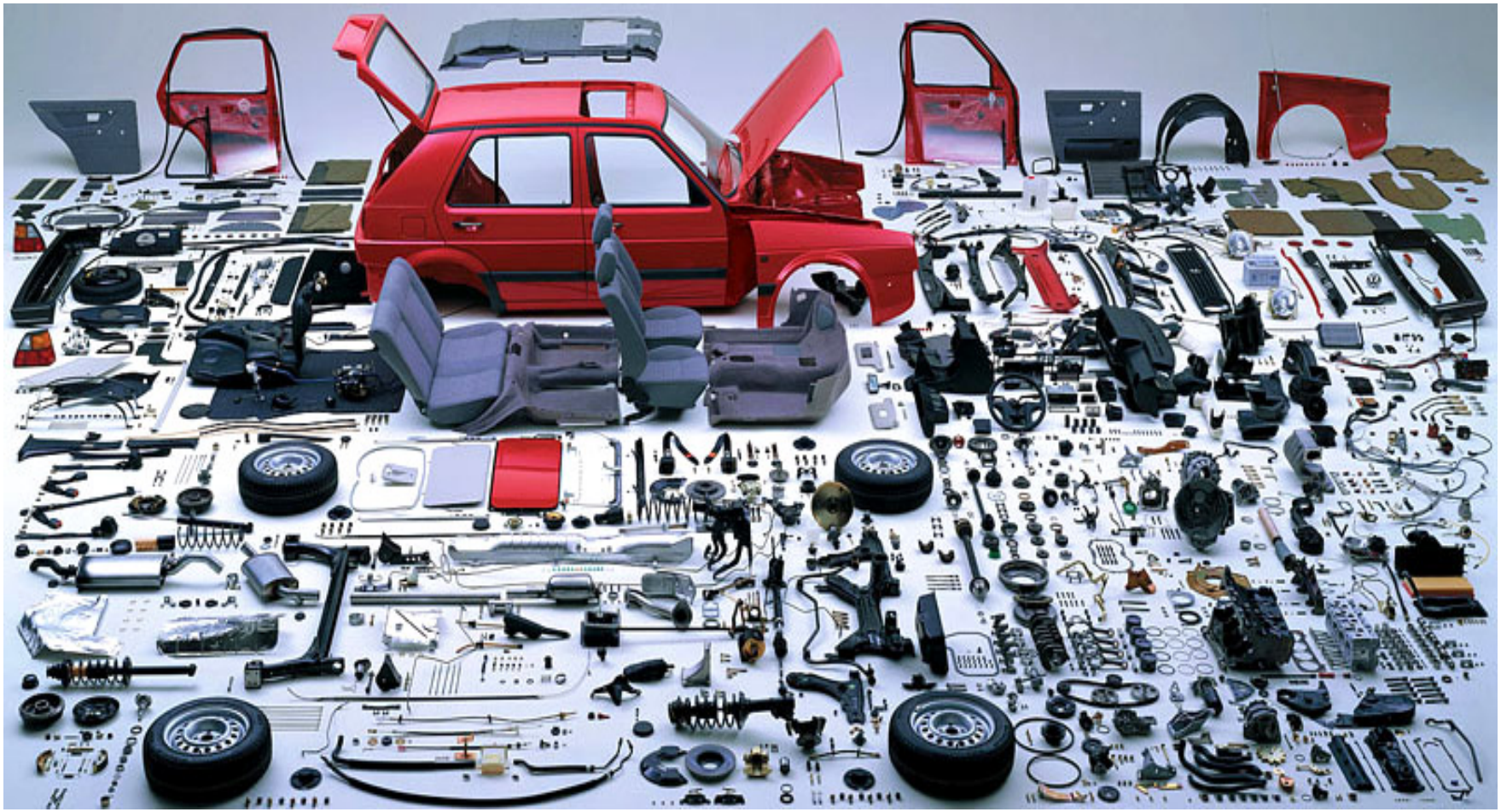
5.2 单元测试的策略

5.3 测试用例的设计(白盒方法)

5.4 单元测试工具

5.5 集成测试

从传统制造业能得到什么启发?



什么是单元测试?

- 单元测试：对已实现的软件最小单元进行测试，以保证构成软件系统的各个单元的质量。
- 单元测试活动中，强调被测试对象的独立性，缩小了问题的分析范围。
- 单元测试应从各个层次来对单元内部算法、外部功能实现等进行检验，包括：
  - 对程序代码的静态评审和分析
  - 运行单元程序以验证其功能特性

单元测试的目标

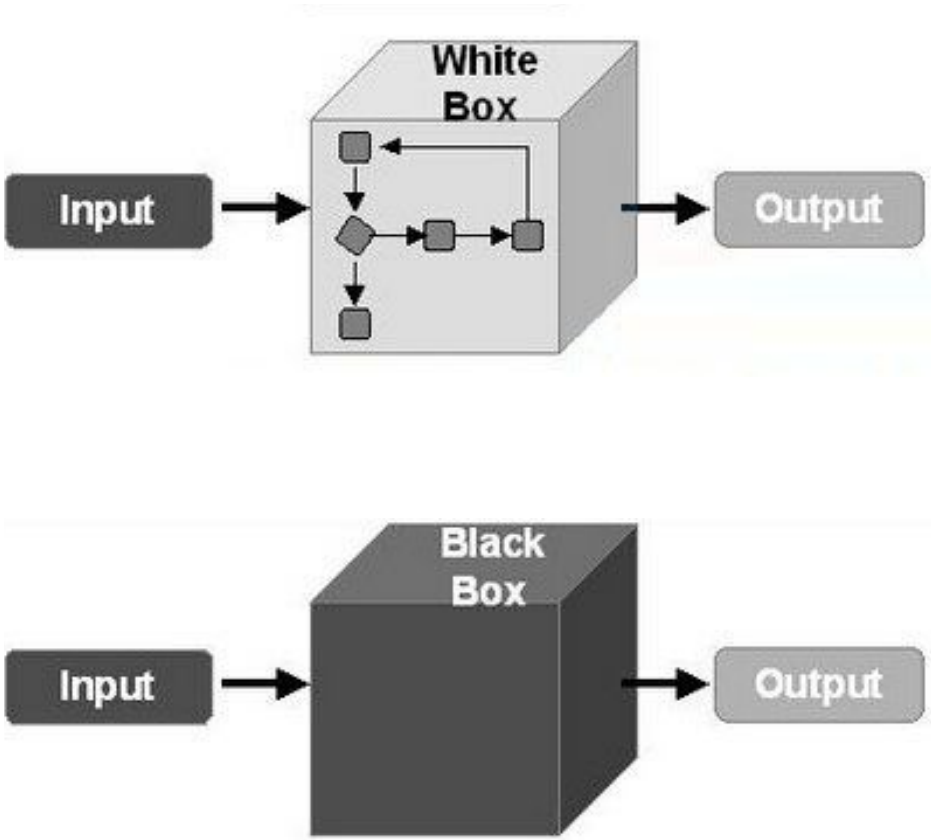
- 被测单元实现了其特定的功能，如果需要，应能返回正确的值；
- 被测单元的运行能够覆盖预先设定的各种逻辑；
- 在被测单元工作过程中，其内部数据能够保持完整，包括全局变量的处理、内部数据的形式、内容及相互关系等不发生错误；
- 被测单元可以接受正确数据，也能处理非法的数据；在数据边界条件上，单元也能够正确工作；
- 被测单元的算法合理，性能良好；
- 被测单元的代码没有发现任何安全性问题。

单元测试的必要性

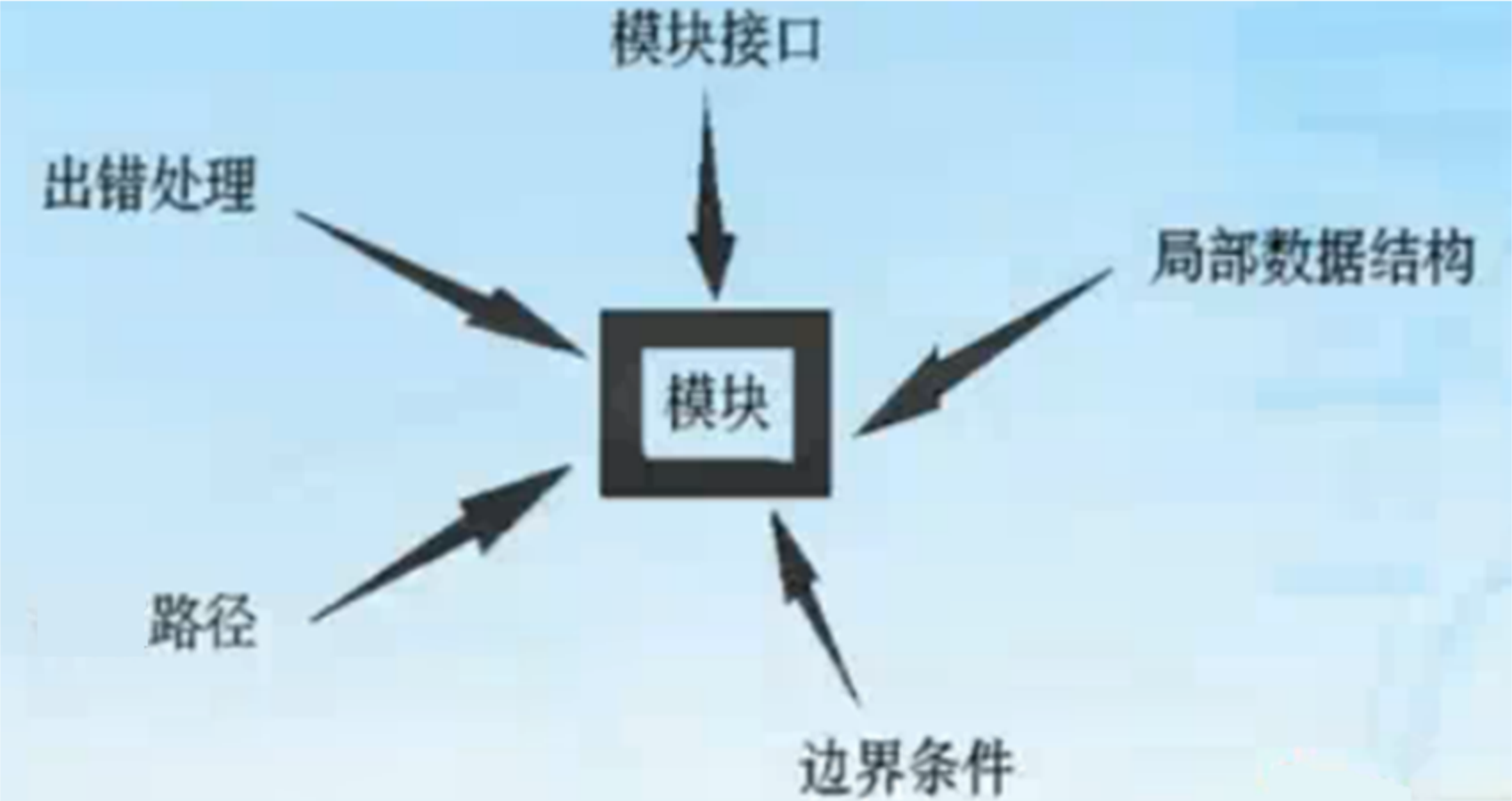
- 通过单元测试可以更早地发现缺陷，缩短开发周期，降低软件成本。
- 单元的质量是整个软件质量的基础，充分的单元测试是很有必要的。

单元测试的方法

- 单元测试主要采用白盒测试方法，辅以黑盒测试方法。
  - 白盒测试方法应用于代码评审、单元程序检验之中。
  - 黑盒测试方法应用于模块、组件等大单元的功能测试之中 。



单元测试要解决的问题



1、对模块接口的测试内容

- 调用被测模块时，其输入参数(实参)与模块的定义参数(形参)在数量、类型和次序上是否匹配?
- 被测模块调用其它模块时，其输入参数(实参)与模块的定义参数(形参)在数量、类型和次序上是否匹配?
- 被测模块内部是否修改了只作输入的形参?
- 全局变量的定义在各模块是否一致?
- 对于非被测模块内部定义的变量（主要是指全局变量、采用地址传递方式的形参），其使用否合理?

对模块接口的测试内容（续）

- 若被测模块需要使用外部资源时，还应测试：
  - 是否检查了该资源的可用性?
    - 资源是否存在?
    - 资源的属性是否允许读或写?
  - 打开资源时的方式是否正确?
    - 二进制？文本?
    - 新建？只读？追加?
  - 开辟的数据缓冲区容量与读取到的数据长度是否匹配?
  - 资源使用结束后有没有及时关闭或释放?
  - 对于I/O的错误是否有检查和处理?

2、对局部数据结构的测试内容

- 是否出现了不正确或不一致的变量声明?
- 变量的命名是否符合规范要求？是否冲突?
  - 即使不会出现问题，也最好不要使用外部变量名来定义内部变量。
- 对于变量的初始值或缺省值是否有错?
- 是否使用了尚未声明的变量?
- 是否使用了尚未赋值或尚未初始化的变量?
- 是否存在声明了却没引用过的变量?
- 变量精度够吗？类型转换会否溢出?
- 数组元素的下标是否正确？数组是否越界?
- 会否出现非法指针?

3、对边界条件的测试内容

- 边界条件主要是指变量的临界数据。
- 需要考虑的变量包括：
  - 输入或输出的业务数据变量
  - 要进行比较判断的变量
  - 循环的控制变量
- 对边界条件的测试内容包括：
  - 普通合法数据是否正确处理?
  - 普通非法数据是否正确处理?
  - 边界内最接近边界的合法数据是否正确处理?
  - 边界外最接近边界的非法数据是否正确处理?

等价类划分

边界值分析

4、对路径的测试内容

单元测试的基本任务是保证模块中每条语句至少执行一次，因此应对被测模块的每一条独立执行路径进行测试。

基本路径法

- 对执行路径进行测试可以发现计算错误、判定错误、控制流错误。

- 计算错误包括：

死代码	错误的计算优先级
算法错误	初始化不正确
精度错误	表达式的不正确符号

对路径的测试内容（续）

❖ 针对比较判定和控制流的错误包括：

- 不同数据类型的比较
- 不正确的逻辑运算符或优先次序
- 浮点运算造成的两值比较不等
- 关系表达式中出现错误的变量或比较符
- “差1错”：即多循环或少循环一次
- 错误的或不可能的循环终止条件
- 当遇到发散的迭代时不能终止循环
- 不适当地修改了循环变量

5、对出错处理的测试内容

- 一个好的程序设计应能预见各种出错条件，并预设适当的出错处理。
- 对出错处理的测试内容应包括：
  - 是否检查了错误出现?
  - 若出现错误，是否进行了错误处理?
  - 错误处理是否有效?
  - 在对错误进行处理之前，错误条件是否已引起系统的干预？对错误条件的处理是否正确?
  - 对出错的描述是否容易理解？是否能帮助定位出错处?
  - 显示的错误信息与实际的错误是否相符?

5.1 什么是单元测试

5.2 单元测试的策略

5.3 测试用例的设计(白盒方法)

5.4 单元测试工具

5.5 集成测试

回顾：单元测试的流程

自动化方式
白盒、静态

人工方式
白盒、静态

自动化方式/
人工方式
白盒、动态

1、代码审查

- 代码审查的目的：为了产生合格的代码，检查源程序编码是否符合详细设计的编码规定，确保编码与设计的一致性和可追踪性。
- 审查的内容：
  - 业务逻辑：代码的实现是否遵照了设计规格说明书？逻辑是否正确且简单？思路是否清晰？
  - 算法的效率：是否还有优化的可能？
  - 代码风格：命名规则、注释、嵌套层数、书写格式、缩写、空行空格等是否符合要求？
  - 编程规则：数据定义、常变量声明、函数的声明和调用、内存处理、表达式等是否正确？

常见的规则/标准

❖ 命名规则：

- 匈牙利命名法（Hungarian）
- 骆驼命名法（camelCase）
- 帕斯卡命名法（PascalCase）

❖ 编程规则：

- MISRA C编程规范
- C++编程规范
- Java编程规范
- ……

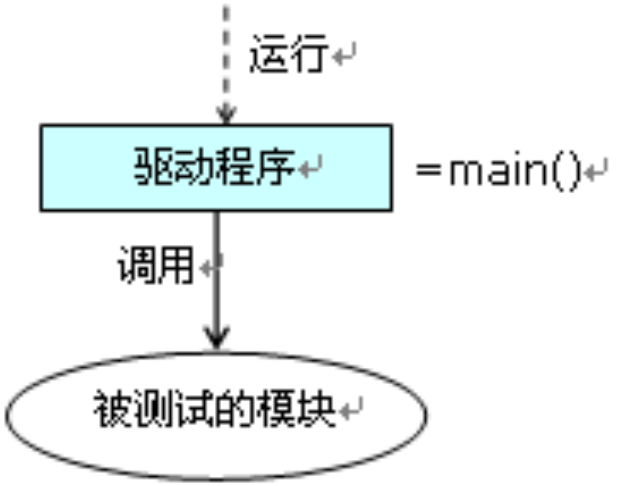
} 各公司自定义

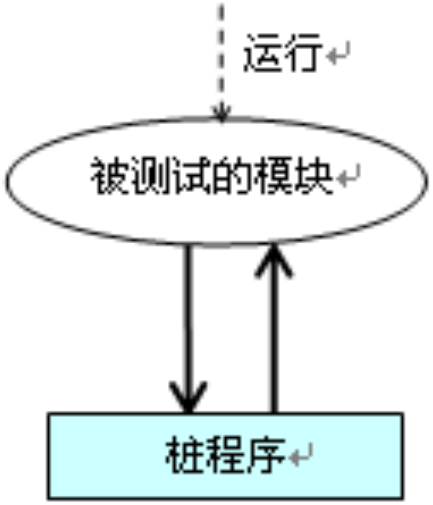
代码评审检查表

- 代码评审的方法：桌面检查、代码审查、走查
- 和需求评审、设计评审一样，为了提高评审的效率，在代码评审环节也可以设定检查表。
- 把程序编码中可能发生的各种缺陷进行分类，为每一类列举尽可能多的典型缺陷，从而形成代码缺陷检查表。
- 示例：一份代码评审检查单。

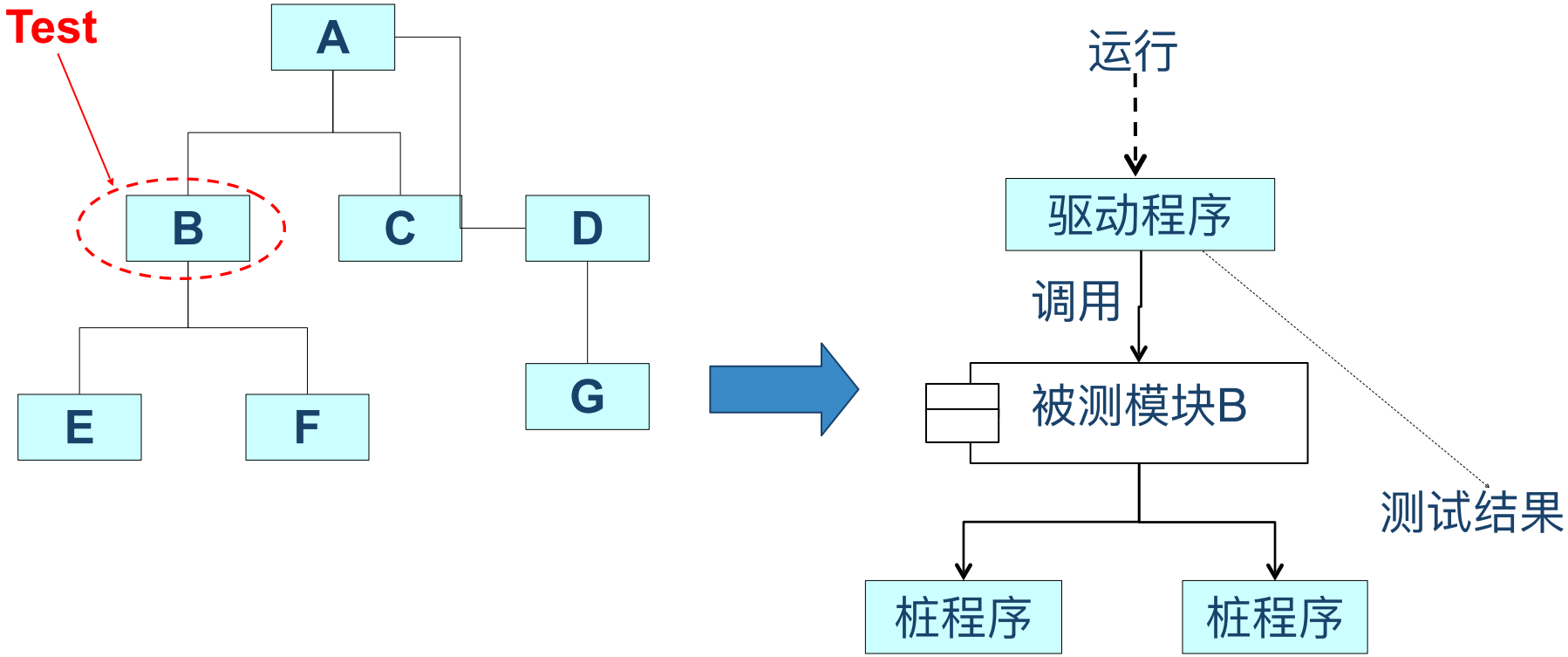
2、单元动态测试的策略

- 在单元测试活动中，强调被测试对象的独立性。
- 但被测模块并不独立，在单元测试时需要使用驱动模块和桩模块来帮助测试。
  - 驱动模块(**driver**)：负责调用被测模块，用以模拟被测模块的上级模块；
  - 桩模块(**stub**)：也称存根程序，将被被测模块调用，用以模拟下层模块。





示例



程序插桩技术

- 插桩：向被测源程序中添加一些语句，以检查程序语句的执行情况、变量的变化情况等。
- 在实际测试中，常在如下位置设立探测点：
  - 程序块的第一个可执行语句之前；
  - 循环语句的出入口；
  - 条件语句的各条分支入口；
  - 输入/输出语句之后；
  - 函数、过程、子程序调用语句之后；
  - **return**语句之后。

单元测试的一些优秀实践

- 最好在软件设计时就编写好单元测试的框架和测试代码。
- 为每个软件单元构建相应的测试代码，并集成到自动测试的框架中，以便于在每日的持续集成时也能运行单元测试。
- 每个单元的测试都应运行在一个干净的测试环境上，每次完成一个测试后要清理测试现场。
- 测试用例或脚本应和代码一起进行版本维护，两者保持修订同步。

5.1 什么是单元测试

5.2 单元测试的策略

5.3 测试用例的设计(白盒方法)

5.4 单元测试工具

5.5 集成测试

PIE模型

静态存在于软件中的缺陷

Fault

Execution
执行Fault处代码

软件执行到Fault处触发的错误

Error

Infection
感染触发出Error的中间状态

Error传播到软件外部，使用户观测到的失效

Failure

Propagation
Error传播，使得观测到的输出结果与预期结果不一致

例：计算数组中所有元素的平均值

```
public static void CSta (int [ ] numbers)
{
    int length = numbers.length;
    double mean, sum;

    sum = 0.0;
    for (int i = 1; i < length; i++)//i=0
    {
        sum += numbers [ i ];
    }
    mean = sum / (double) length;

    System.out.println ("mean:  " + mean);
}
```

Test input:
[3,4,5]

sum=3+4+5

sum=4+5

mean=4

mean=3

但，如果测试用例的数据是……

```
public static void CSta (int [ ] numbers)
{
    int length = numbers.length;
    double mean, sum;

    sum = 0.0;
    for (int i = 1; i < length; i++)//i=0
    {
        sum += numbers [ i ];
    }
    mean = sum / (double) length;

    System.out.println ("mean:  " + mean);
}
```

Test input:
[0,4,5]

sum=0+4+5

sum=4+5

mean=3

mean=3

还有一种情况……

```
public static void CSta (int [ ] numbers)
{
    int length = numbers.length-1;
    double mean, sum;

    sum = 0.0;
    for (int i = 0; i < length; i++)
    {
        sum += numbers [ i ];
    }
    mean = sum / (double) length;

    System.out.println ("mean:  " + mean);
}
```

Test input:
[3,5,4]

sum=3+5+4

sum=3+5

mean=12/3=4

mean=8/2=4

PIE模型（续）

- 从上述的几个例子可以看到：
  - 包含**fault**的代码，不一定会产生错误的中间状态**error**；
  - 产生了错误的中间状态**error**，也不一定会出现失效**failure**。
- 因此，如果只观测输出，将有可能错过**fault**！

该如何设计测试数据，才能更好地发现fault？

回顾：白盒与黑盒

- 白盒测试（结构测试）：不考虑被测程序的对外功能，只根据其内部结构来设计测试用例，要求对被测程序的语句达到一定的覆盖率。
- 黑盒测试（功能测试）：不考虑被测程序的内部结构，只依靠被测程序输入和输出之间的关系，或程序的功能来设计测试用例，主要依据软件需求规格说明书考查功能是否实现。
- 白盒和黑盒是动态测试的主要方法：
  - 白盒测试常用于单元和集成测试。
  - 黑盒测试常用于功能验证、系统和验收测试。

白盒测试

- 白盒测试方法：**White-box Testing**，也称结构测试或逻辑驱动测试。
- 白盒测试方法是根据模块内部的逻辑结构，针对程序语句、路径、变量状态等内容进行测试，检验程序中的各个分支条件是否得到满足、每条执行路径是否按预定要求正确地工作。
  - 逻辑覆盖
  - 基本路径测试
  - 循环覆盖

关注代码，忽视单元的实际功能特性。

白盒动态测试的目标

- 确保没有死代码。
  - 死代码是指在程序运行过程中永远不可能被执行到的代码。
  - 白盒测试要求每条语句都被至少执行一次。
- 确保对所有的逻辑值都测试**true**和**false**。
- 在上下边界和合法的范围内运行所有的循环。
- 确保内部数据结构的有效性和完整性。

- 可见，白盒动态测试对中间状态做了检测，可有效地发现**error**和**failure**，找到**fault**。

白盒动态测试的方法

- 逻辑覆盖法（控制流覆盖）
  - 语句覆盖(行覆盖、段覆盖)
  - 判定覆盖(分支覆盖)
  - 条件覆盖
  - 条件判定覆盖
  - 条件组合覆盖(多条件覆盖)
  - 路径覆盖
- 基本路径法
- 循环覆盖法

弱 → 强

发现错误的能力

1、语句覆盖

- 语句覆盖(行覆盖)：让被测代码中的每条可执行语句都至少执行一次。

被测代码：

```
int foo(int a, int b)
{
return  a / b;
}
```

测试用例：

a=10，b=5

此时的语句覆盖率达100%，但不能发现除零异常。

- 有时无法做到100%的语句覆盖，如逻辑组合不成立（a>b&&b>a）、处理异常的代码等。

语句覆盖（续）

- 语句覆盖是“最弱的覆盖”，它只管覆盖代码中的执行语句，却不考虑语句的分支。

被测代码：

```
int f1(int a, int b)
{
  if (a>=0)
    a=a+b;
  b=a;
  return  b;
}
```

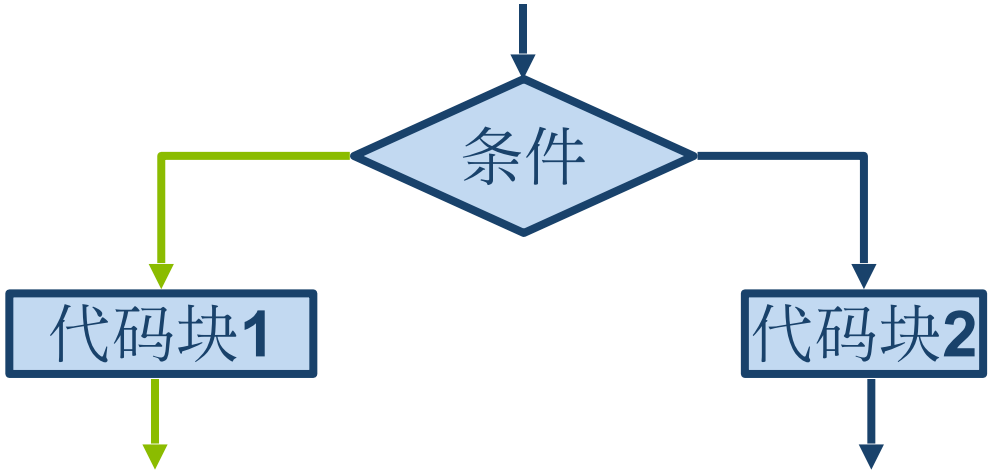
测试用例：

a=10，b=5

仅使用了一个测试用例就将语句的覆盖率达到了100%，但没能测试到a<0的情况。

2、分支覆盖

- 分支覆盖(判定覆盖)：对每个条件判断的真假两路分支各至少执行一次。



- 分支覆盖可确保语句覆盖，但它将用于判断的表达式作为整体，即便是由与或组成的复合条件，也只考虑整个表达式对外是真还是假。

分支覆盖（续）

被测代码：

```
int f2(int a, int b)
{
  if ((a>0)&&(b>0))
    return  a;
  else
    return  a+b;
}
```

测试用例：

①a=1，b=1　T

②a=-1，b=1　F

分支	逻辑取值情况	测试用例
T	a>0，b>0	①
F	a>0，b<=0	无
	a<=0，b>0	②
	a<=0，b<=0	无

上述两个测试用例虽然覆盖了真假两个分支，但没有测试到b<=0的情况。

3、条件覆盖

❖ 条件覆盖：让条件判断中的每个最小的条件至少取一次真和一次假。

被测代码中的条件判断：(a>0)&&(b
有2个最小条件，4种取值：
a>0、a<=0、b>0、b<=0

有没有办法同时覆盖所有的条件和所有的分支？

测试用例：
①a=1，b=-1
②a=-1，b=1

上述两个用例覆盖了条件，但没覆盖到T分支。

分支	逻辑取值情况	测试用例
T	a>0，b>0	无
F	a>0，b<=0	①
	a<=0，b>0	②
	a<=0，b<=0	无

4、分支条件覆盖

❖ 分支条件覆盖：同时覆盖分支和条件。

被测代码中的条件判断：**(a>0)&&(b>0)**
有**2**个最小条件，**4**种取值：
a>0、**a<=0**、**b>0**、**b<=0**

测试用例：
①a=1，b=1
②a=-1，b=-1

上述两个用例覆盖了分支和条件，但没覆盖到所有组合情况。

分支	逻辑取值情况	测试用例
T	a>0，b>0	①
F	a>0，b<=0	无
	a<=0，b>0	无
	a<=0，b<=0	②

5、条件组合覆盖

❖ 条件组合覆盖：覆盖所有逻辑条件的组合。

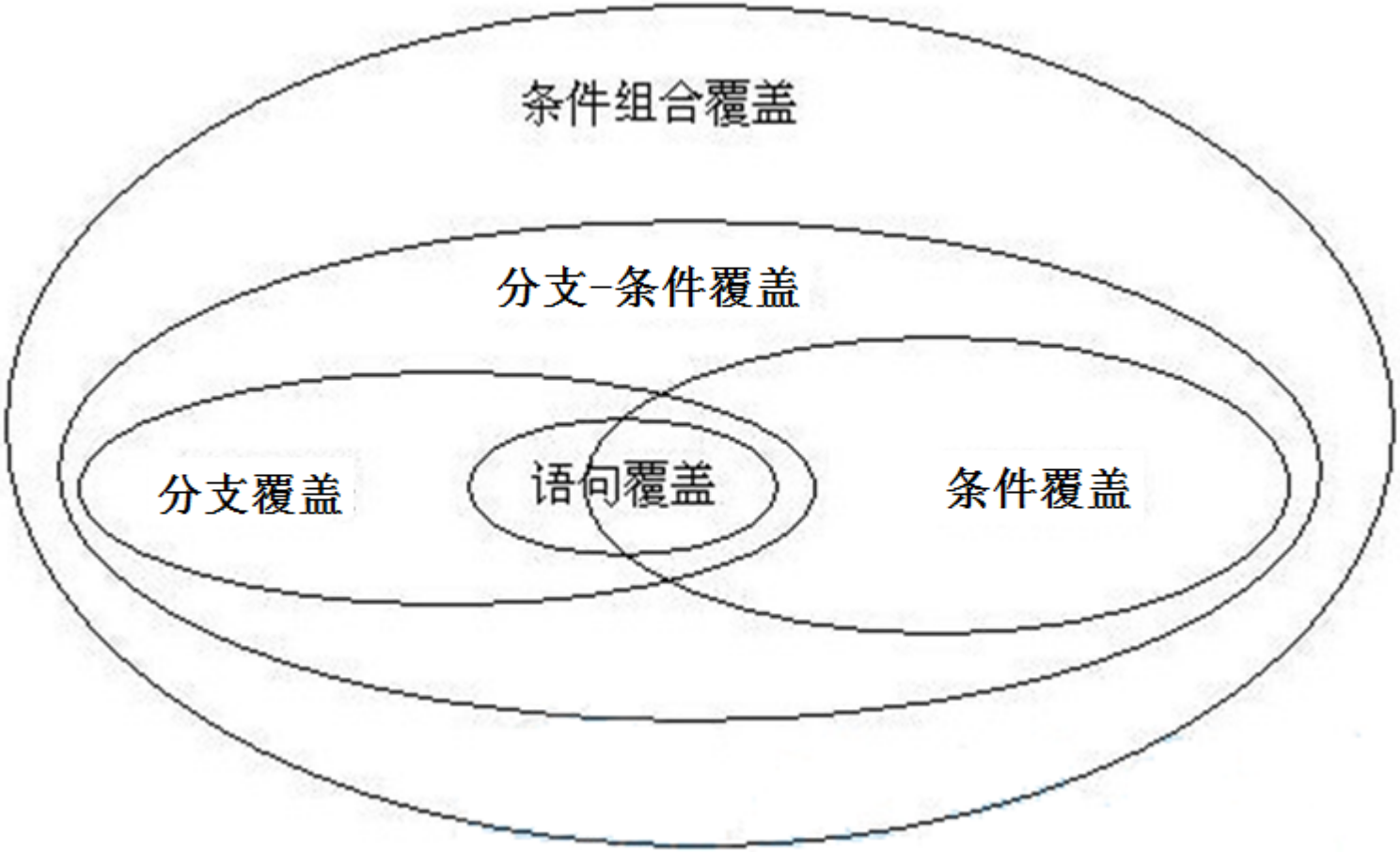
被测代码中的条件判断：(a>0)&&(b>0)
有2个最小条件，4种取值：
a>0、a<=0、b>0、b<=0

测试用例：
①a=1，b=1
②a=1，b=-1
③a=-1，b=1
④a=-1，b=-1

分支	逻辑取值情况	测试用例
T	a>0，b>0	①
F	a>0，b<=0	②
	a<=0，b>0	③
	a<=0，b<=0	④

各条件组合都覆盖了，但测试用例数量多了一倍！

条件组合覆盖（续）



❖ 问题：条件组合覆盖是不是真的“全覆盖”了？

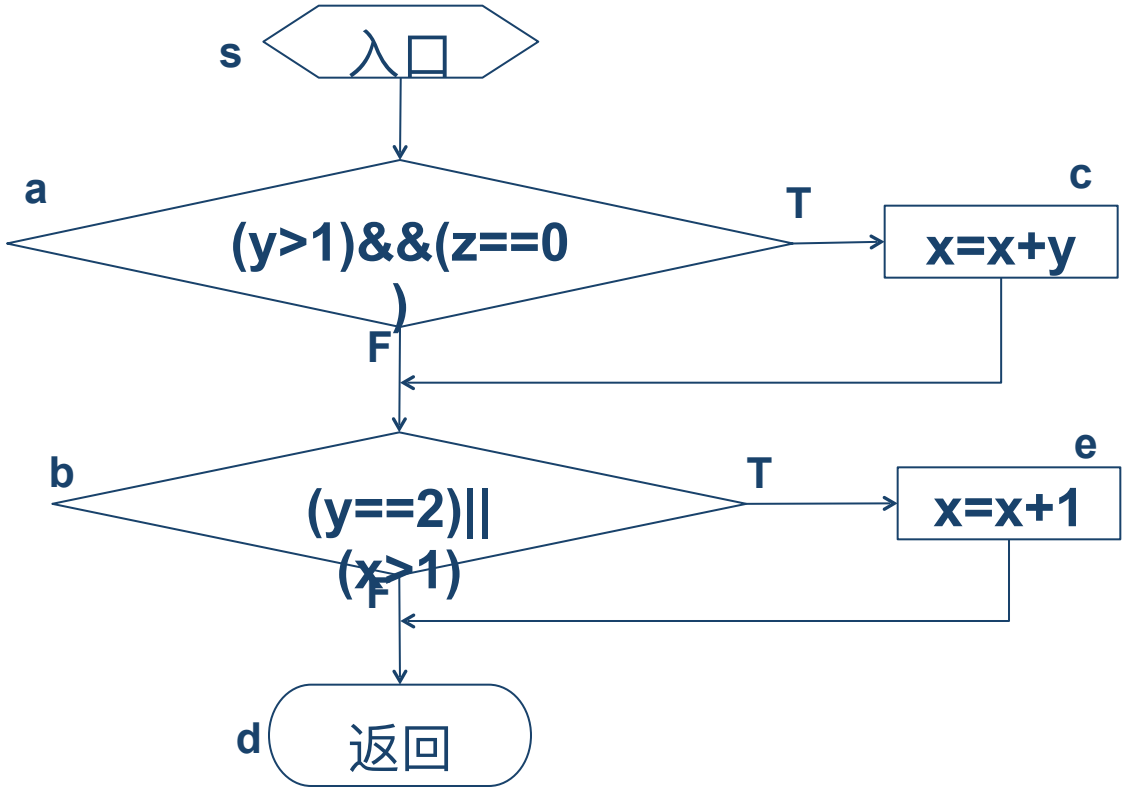
6、路径覆盖

- 什么是路径?
- 路径是图论中的概念。

若G中的结点序列$p = <v_1, v_2, \ldots v_k>$，满足对于所有的$j$,有$(v_{i+j}, v_{i+j+1}) \in E, (0 \le j < k-i)$，则称$p$为程序的一条子路径。若$v_1 = v_k$，则称$P$为一条回路（环路）。若子路径$p$从源结点$I$到汇结点$O$，则称$p$为程序的一条路径。

- 将程序使用图的形式(程序流程图、流图)表示，再来判断所有路径是否覆盖。

路径覆盖示例



判定a			
①	y>1	z==0	T
②	y>1	z<>0	F
③	y<=1	z==0	F
④	y<=1	z<>0	F
判定b			
⑤	y==2	x>1	T
⑥	y==2	x<=1	T
⑦	y<>2	x>1	T
⑧	y<>2	x<=1	F

四条路径：sabd，sacbd
sabed，sacbed

❖ 若按条件组合覆盖设计测试用例：

测试用例	覆盖的条件组合	所行经的路径
x=4，y=2，z=0	①⑤	sacbed
x=1，y=2，z=1	②⑥	sabed
x=2，y=1，z=0	③⑦	sabed
x=1，y=1，z=1	④⑧	sabd

路径sacbd没能覆盖到！

❖ 按路径覆盖设计测试用例：

测试用例	覆盖的条件组合	所行经的路径
x=4，y=2，z=0	①⑤	sacbed
x=3，y=3，z=0	①⑧	d
x=2，y=1，z=0	③⑦	d
x=1，y=1，z=1	④⑧	sabd

条件②⑥没能覆盖到！

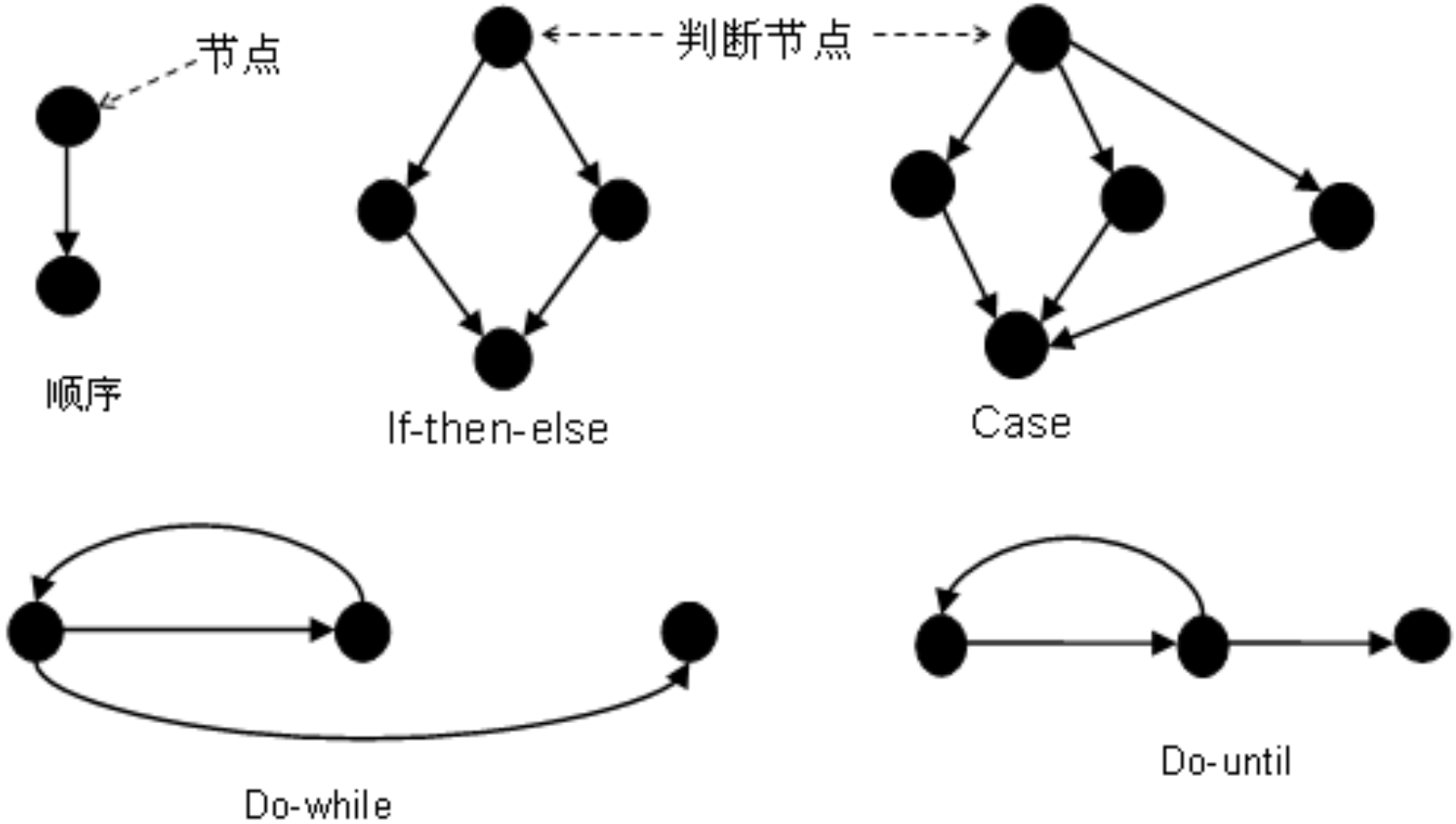
7、基本路径法

- 路径覆盖仍不完善，从图论的角度来讨论，提出了基本路径测试方法。
- 基本路径法的步骤：
  ① 根据程序或流程图绘制流图；
  ② 计算其环复杂性；
  ③ 确定一组独立的程序执行路径(即基本路径，其集合称为基本集)；
  ④ 为每条基本路径设计一个测试用例。
- 执行该基本集所生成的测试用例，能保证程序中的每一条可执行语句至少被执行一次。

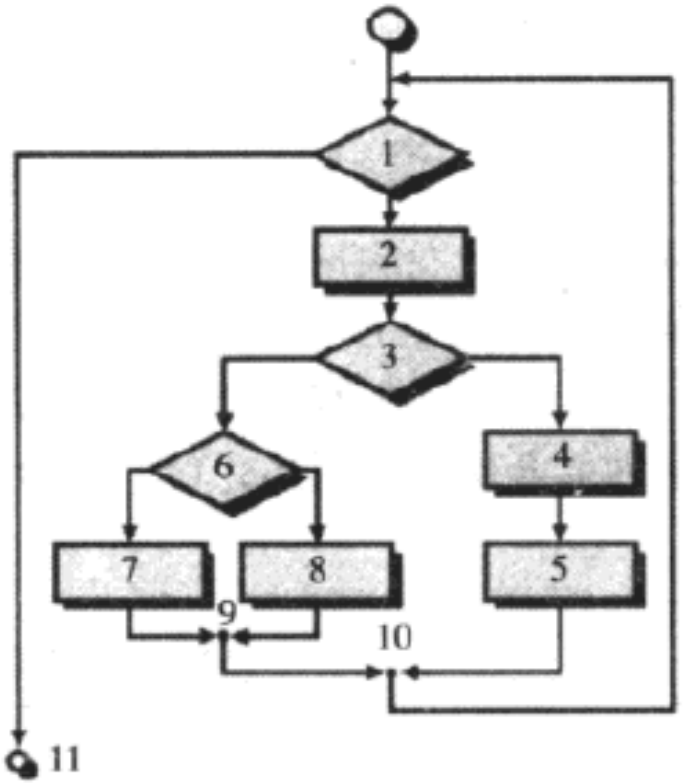
流图

- 流图：利用简单符号描述程序的逻辑控制流。
  - 圆：结点，表示一个或多个过程语句。程序流程图中的处理框序列和一个菱形判定框可映射为单个结点。
  - 箭头：边，表示控制流。一条边必须终止于一个结点，即使该结点并不代表任何过程语句(空结点)。
  - 由边和结点围成的区间称为域，图形外部也是一个域。
- 每种结构化的结构有相应的流图符号。

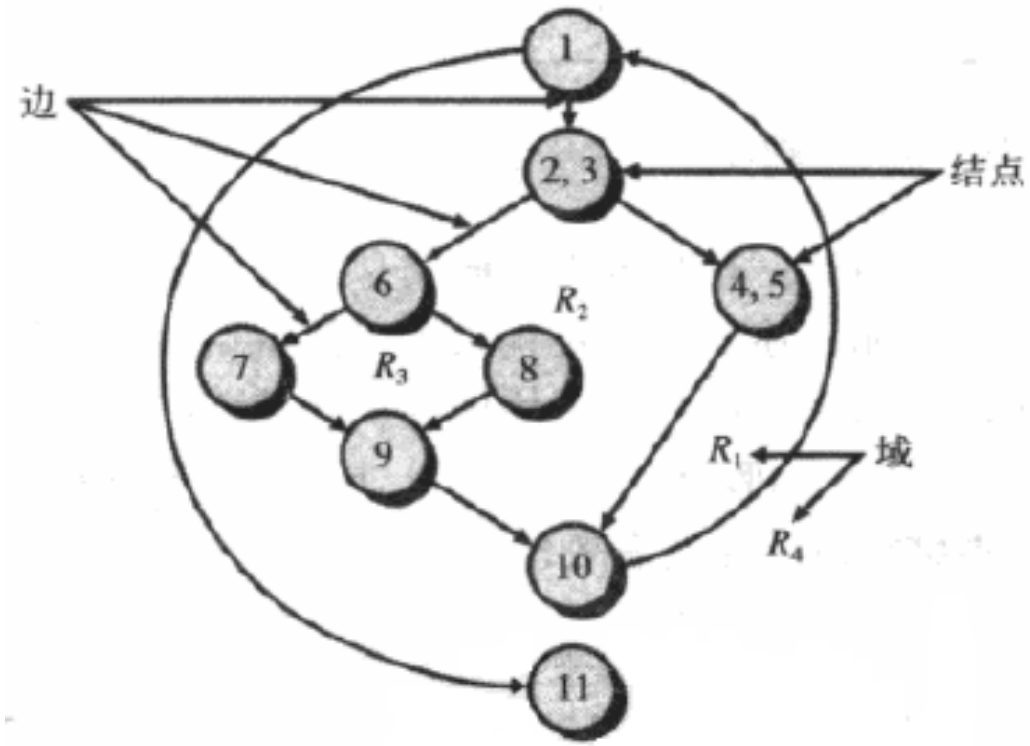
流图中的基本结构



示例：程序流程图到流图-简单条件



a) 流程图

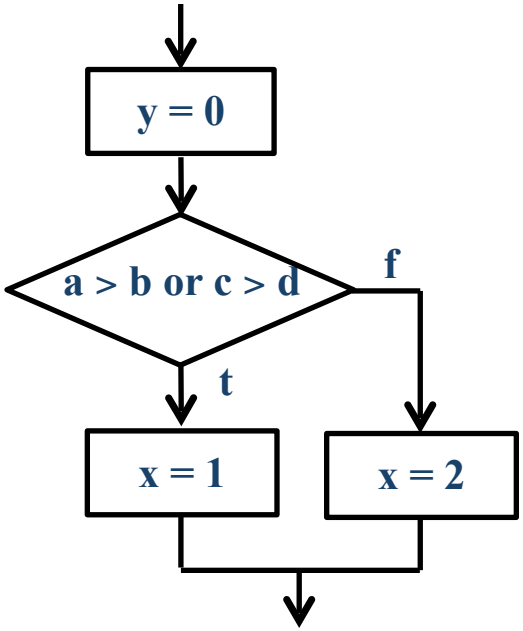


b) 流图

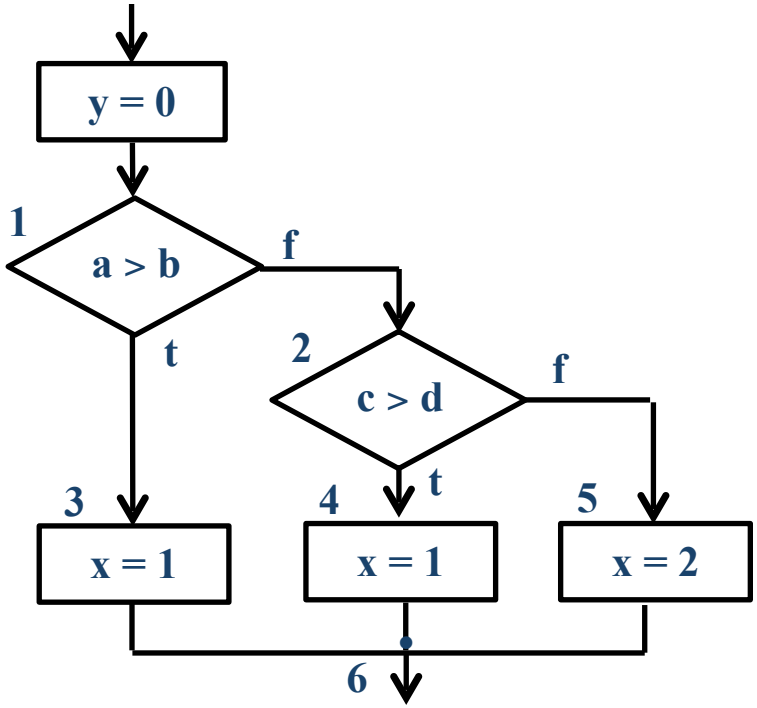
注意：顺序执行的语句可合并；确保图中只有一个入口和一个出口，若不符合，可增加空结点。

示例：程序流程图到流图-复合条件

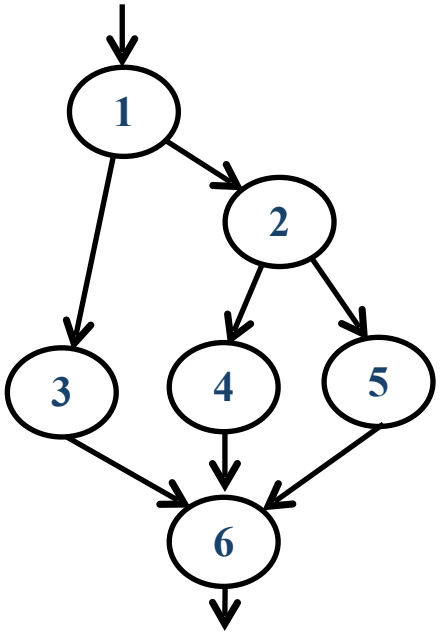
- 如果设计图的判定中包含了复合条件，那么必须先将其转换成等价的简单条件设计图。



a）含复合条件的流程图



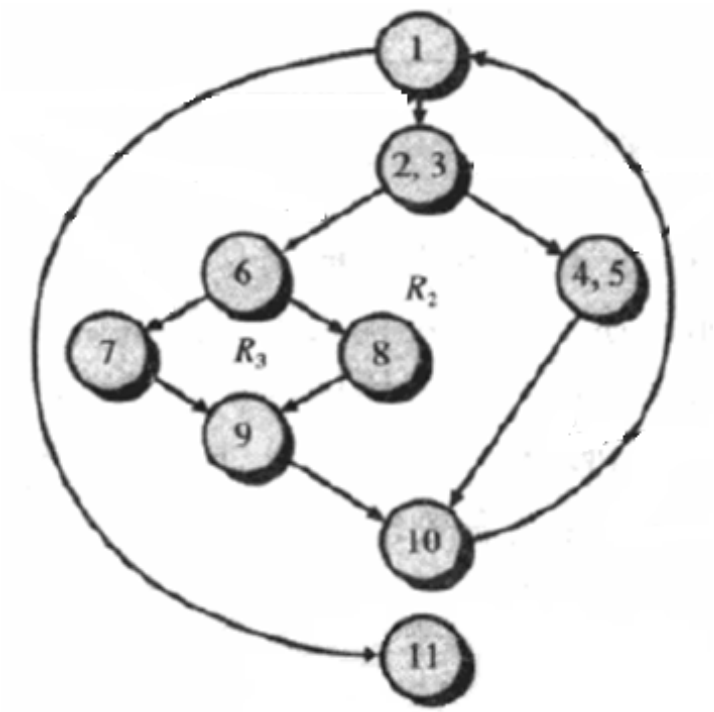
b）只含简单条件的流程图



c）对应的流图

独立路径和环复杂性

- 独立路径是任何贯穿程序的、至少引入一组新的处理语句序列或一个新条件的路径；在流图中，独立路径至少包含一条在定义该路径之前未曾用到过的边。
- 环复杂性以图论为基础，有三种计算方法：
  1. V(G) = 域的数量
  2. V(G) = 边数 - 结点数 + 2
  3. V(G) = 判定结点数 + 1
- 环复杂性指示了独立路径的条数。



该流图的环复杂性可如下计算：

方法1：V(G)=域的个数=4

方法2：V(G)=E-N+2=11-9+2=4

方法3：V(G)=P+1=3+1=4

图中的一组独立路径如下：

路径1：1-11

路径2：1-2-3-4-5-10-1-11

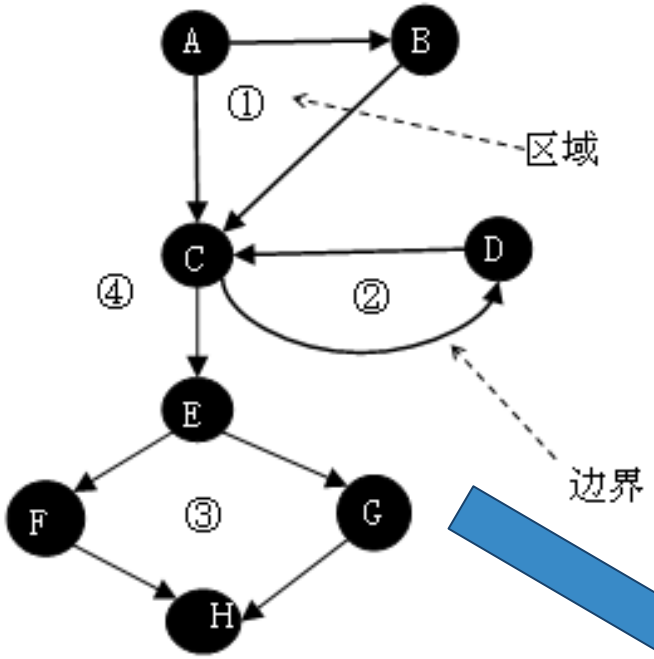
路径3：1-2-3-6-8-9-10-1-11

路径4：1-2-3-6-7-9-10-1-11

最后，为基本路径集中的每条独立路径设计一个测试用例，以确保程序执行对应的路径。

注意：
基本集不是唯一的！

图矩阵



可以将流图转换成矩阵，用以辅助分析和评估。

	A	B	C	D	E	F	G	H	
A		1	1						判定节点
B			1						
C				1	1				判定节点
D			1						
E						1	1		判定节点
F								1	
G								1	
H									

图中的结点按次序排成行列，矩阵的每行表示从该结点出发能达到哪些结点。

图矩阵的元素：连接权值

- 图矩阵中的元素被称为连接权值，它可为流图补充附加信息。
- 最简单情况下，连接权值用1表示存在连接、用0表示不存在连接。
- 连接权值还可以赋予其它属性，以帮助评估代码质量：
  - 执行连接(边)的概率
  - 穿越连接的处理时间
  - 穿越连接时所需的内存
  - 穿越连接时所需的资源

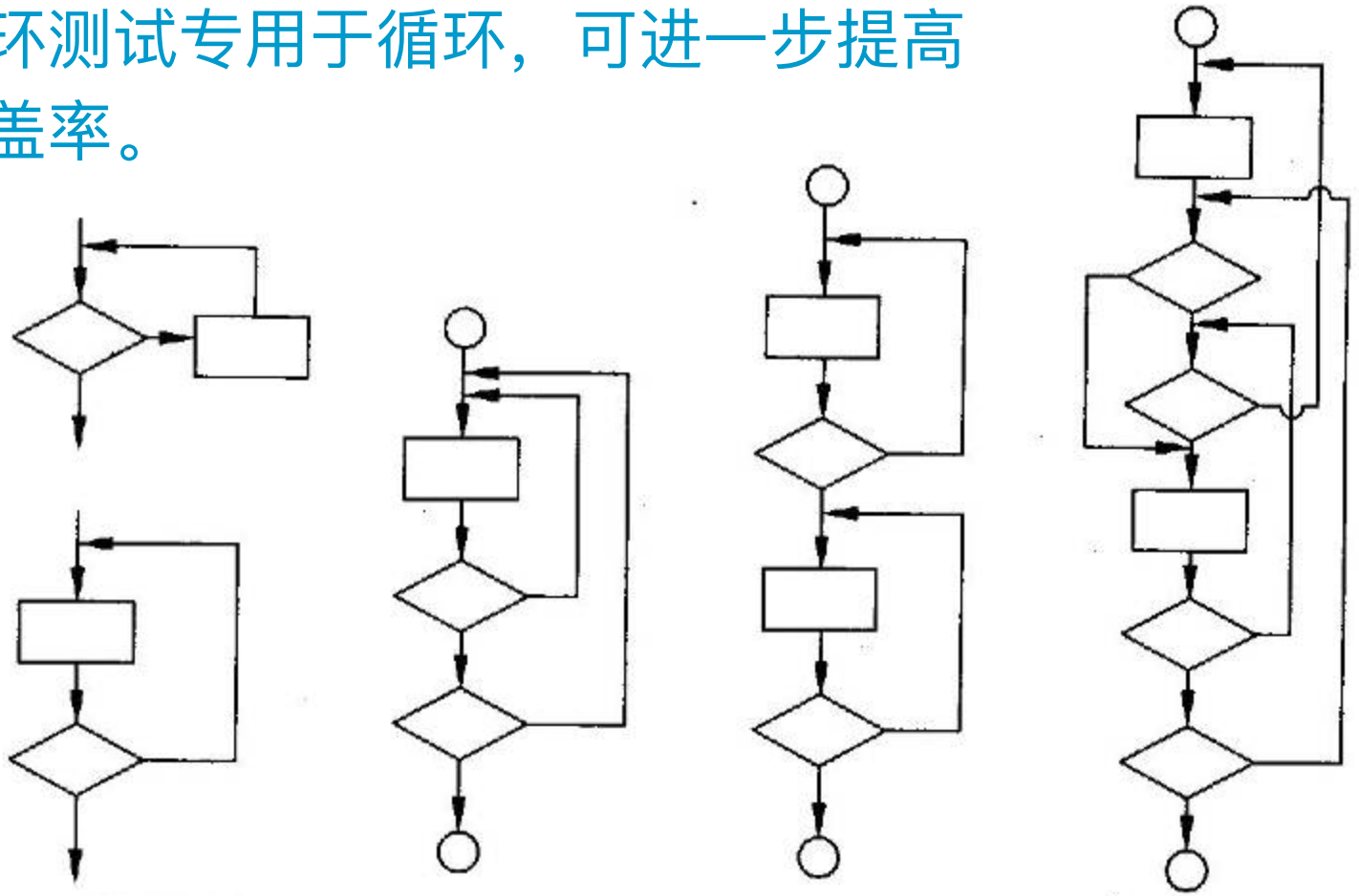
作业

- 使用基本路径法为右侧的程序设计测试用例，要求：
  - 将源代码绘制成程序流程图
  - 将程序流程图绘制成流图
  - 计算流图环复杂性
  - 列出基本路径集
  - 设计测试用例

```
void  Sort(int iRecordNum,int iType)
{
    int x=0;
    int y=0;
    while (iRecordNum-- > 0)
    {
        if(0= =iType)
         { x=y+2; break; }
        else
            if (1= =iType)
          x=y+10;
            else
              x=y+20;
    }
}
```

8、循环覆盖

- 循环测试专用于循环，可进一步提高覆盖率。



(a) 简单循环　(b) 嵌套循环　(c) 串接循环　(d) 非结构循环

① 简单循环的测试集：(n为最大循环次数)

a) 零次循环(跳过循环)：从循环入口到出口
b) 1次循环、2次循环、m次循环(m<n)
c) n-1次循环、n次循环、n+1次循环

② 嵌套循环测试集的设计方法/步骤：

a) 从最内层循环开始，将外层循环的迭代次数设置成最小值，对最内层循环执行简单循环测试；
b) 由内向外，测试上一层循环：将该测试层的所有外层循环的迭代次数取最小值，所有内层循环的迭代次数取典型值，对该层按简单循环测试；
c) 重复上一条规则，直到所有各层循环测试完毕。

③ 串接循环的测试方法：

- 如果串接的各个循环互相独立，则可以分别用简单循环的方法进行测试；
- 如果两个循环串接，且第一个循环的循环计数是第二个循环的初始值，即两个循环不独立，此时应把第一个循环看作外循环，第二个循环看作内循环，然后再用嵌套循环的测试办法来处理。

④ 对于非结构化的循环，应先将其重新设计成结构化的程序，然后再测试。

5.1 什么是单元测试

5.2 单元测试的策略

5.3 测试用例的设计(白盒方法)

5.4 单元测试工具

5.5 集成测试

单元测试工具的种类

- 代码规则/风格检查工具
- 内存资源泄漏检查工具
- 代码覆盖率检查工具
- 代码性能检查工具

- 静态测试工具不需要运行代码，而是直接对代码进行语法扫描和所定义的规则进行分析，找出不符合编码规范的地方，给出错误报告和警告信息。
- 动态测试工具则需要通过运行程序来检测程序，需要写测试脚本或测试代码来完成分支覆盖、条件覆盖或基本路径覆盖的测试。

单元测试工具列表

类别	工具
C 语言	C++ Test、CppUnit、QA C/C++、CodeWzard、Insure++6.0
Java 语言	Jtest、Junit、Jmock、EasyMock、MockRunner
JUnit 扩展框架	TestNG 、JWebUnit 和 HttpUnit
GUI （功能）	JFCUnit、Marathor
通用的	Rexelint、Splint、McCabe QA、CodeCheck、GateKeeper
.NET	.TEST、Nunit
Data Object，DAO	DDTUnit，DBUnit
EJB	MockEJB 或者 MockRunner
Servlet，Struts	Cactu，StrutsUnitTest
XML	XMLUnit
内嵌式系统等	Logiscope、JTestCase

开源的单元测试工具

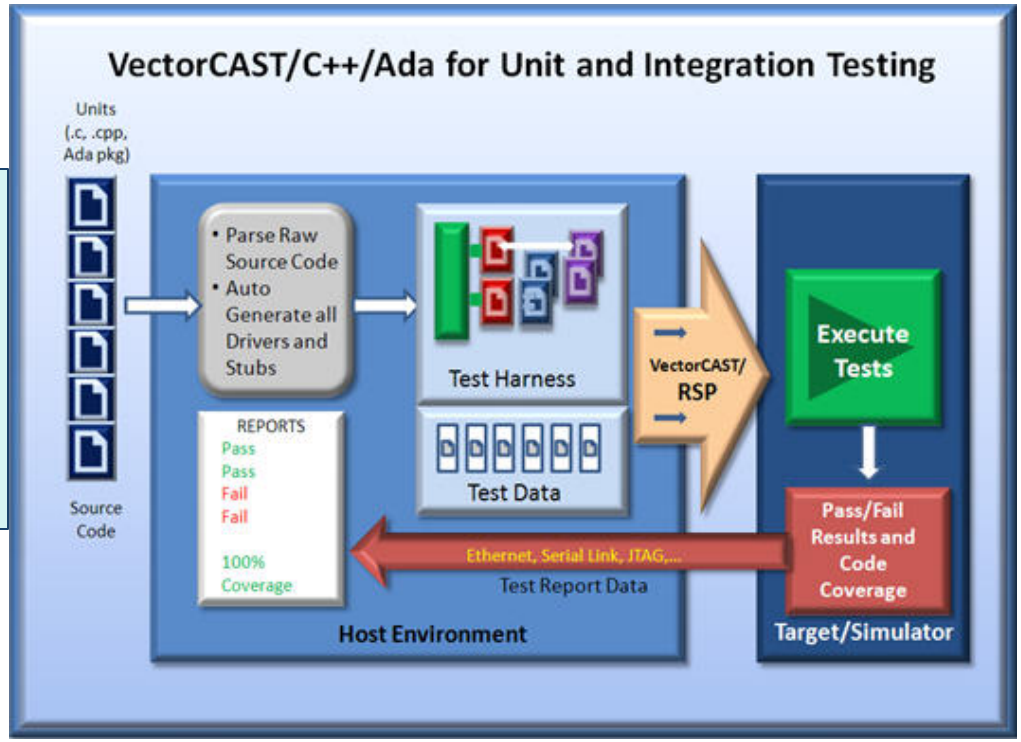
- C/C++ 语言单元测试工具：
  - CppTest、CppUnit、…
- Java语言单元测试工具：
  - TestNG、PMD、Checkstyle、Findbugs、Jalopy……
- Mock Object类工具：
  - MockObjects、Xdoclet、EasyMock、MockCreator、MockEJB、ObjcUnit、jMock等
- 其它

商业性的单元测试工具

- C/C++语言的单元测试工具：
  - Parasoft C++、PR QA•C/C++、CompuWare DevPartner for Visual C++ BoundsChecker Suite、Panorama C++等
- 内存资源泄漏检查工具：
  - CompuWare BounceChecker， IBM Rational PurifyPlus 等
- 代码覆盖率检查工具：
  - CompuWare TrueCoverage， IBM Rational PureCoverage，TeleLogic Logiscope等
- 代码性能检查工具：
  - Logiscope和 Macabe等

单元测试工具介绍

- ***JUnit***
- 微软***VSTS***



1、JUnit

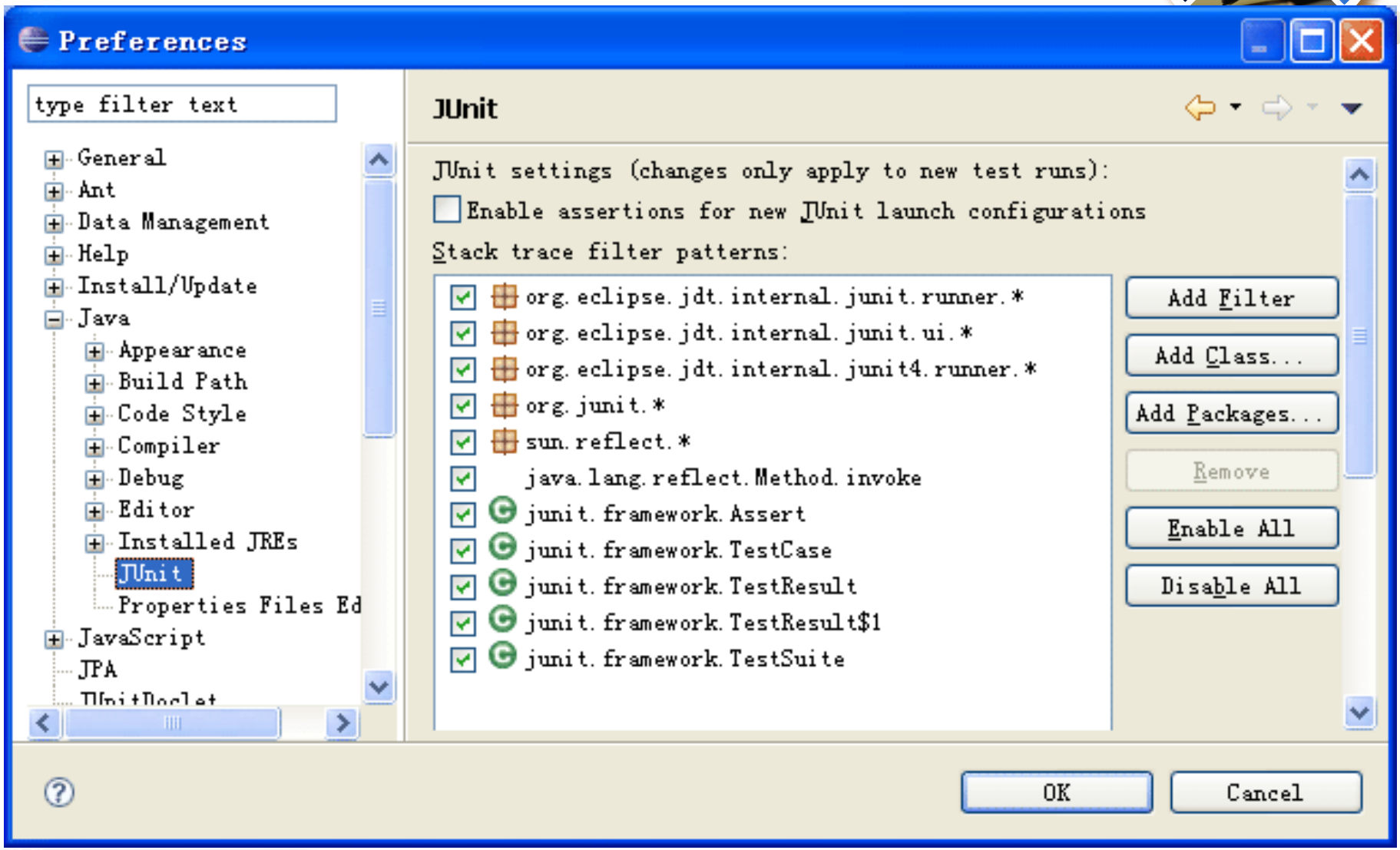
- xUnit系列测试框架为许多常用编程语言提供了各自的单元测试工具，常用的有：
  - JUnit —— Java
  - CppUnit —— C
  - DUnit —— Delphi
  - NUnit —— .net
  - PhpUnit —— Php
- 其中，JUnit是最为成功的一个实例。
- 开发人员只需要按照JUnit的约定编写测试代码，就可以对自己要测试的代码进行测试。

Junit的优势

❖ JUnit的优势：

- 开源，具有很强的扩展性
- 完全免费
- 使用方便，通过编程实现测试
- 测试代码和产品代码分离，有利于测试用例和产品代码的管理
- 提供检验和反馈机制
- 允许组合多个测试用例，并能自动执行
- 能与多种**IDE**环境集成

Eclipse中的JUnit



Java - Java project one/src/testSampleCalculator.java - Eclipse Platform

File Edit Source Refactor Navigate Search Project Run Window Help

Package Explore | Hierarchy | JUnit

Finished after 0.25 seconds

Runs: 2/2 Errors: 0 Failures: 1

testSampleCalculator [Runner: JUnit 4] (0.062 s)
- testAdd (0.047 s)
- testSubtration (0.000 s)

Failure Trace

junit.framework.AssertionFailedError: expected:<72>
at testSampleCalculator.testAdd(testSampleCalculato

JUnit

testSampleCalculator | SampleCalculator.jav

```
public class testSampleCalculator extends TestCase
{
    @Before
    public void setUp() throws Exception {
    }

    @After
    public void tearDown() throws Exception {
    }

      public void testAdd()
      {
            SampleCalculator calculator = new SampleCa
            int result = calculator.add(50 , 20);
             assertEquals(72 , result);
      }

      public void testSubtration()
      {
            SampleCalculator calculator = new Sample
            int result = calculator.subtration(50 , 
            assertEquals(30 , result);
      }

}
```

JUnit的组成

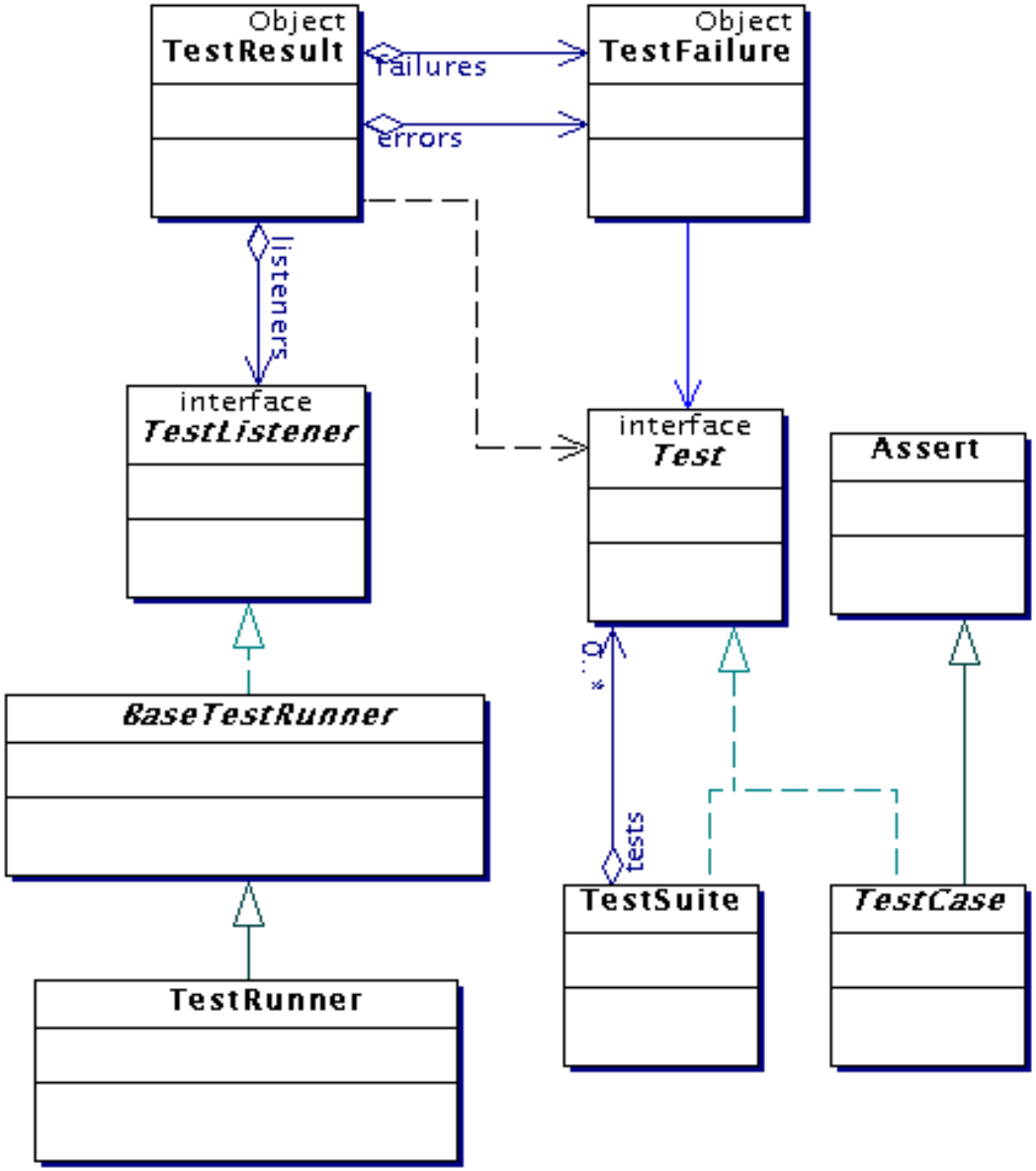
- JUnit以jar文件的形式发布。
- 目前常用的版本是JUnit3.x和JUnit4.x，但两个系列的差别较大。
- JUnit4.12提供了4个包：
  - junit.extensions：扩展包
  - junit.framework
  - junit.runner
  - junit.textui：结果显示界面

  } 核心包
- JUnit有7个核心类。

JUnit的核心类

- **Assert**类：验证类，定义了一组进行比较验证的函数，当比较不成立时抛出异常。
- **Test**接口：测试接口，是各个测试相关类的共同接口，用于实施测试和收集测试结果。
- **TestCase**类：测试用例类，是**Test**接口的抽象实现，但由于是抽象的，因此必须被继承。
- **TestSuite**类：测试套件类，是Test接口的实现，由几个**TestCase**或其它**TestSuite**组成，形成树形结构的测试任务，在同一个线程中依次执行。
- **TestResult**类：结果类，并将结果转发**TestListenter**。
- **TestRunner**：是客户对象调用的起点，负责跟踪整个测试过程，能够显示测试结果，并且报告测试的进度。
- **TestListenter**接口：对测试结果进行处理。

JUnit核心类的关系图



Assert提供的核心方法

assertEquals()	用来查看对象中存的值是否是期待的值，与字符串比较中使用的equals()方法类似；
assertFalse() assertTrue()	用来查看变量是是否为false或true，如果assertFalse()查看的变量的值是false则测试成功，如果是true则失败，assertTrue()与之相反；
assertSame() assertNotSame()	用来比较两个对象的引用是否相等和不相等，类似于通过“==”和“!=”比较；
assertNull() assertNotNull()	用来查看对象是否为空和不为空;
fail ()	让测试失败，并给出制定信息。

哈爾濱工業大學 HARBIN INSTITUTE OF TECHNOLOGY
软件工程系

TestCase提供的方法

countTestcases()	计算run(TestResult result)所执行的TestCase的数目（由Test接口规定）
createResult()	创建默认的TestResult对象
getName()	获得TestCase的名字
run()	运行TestCase并收集TestResult中的结果
runBare()	运行测试序列，但不执行任何特殊功能，比如自动发现test方法
runTest()	重载以运行测试并断言其状态
setName()	设置TestCase的名字
setUp()	初始化fixture
tearDown()	销毁fixture

哈爾濱工業大學 HARBIN INSTITUTE OF TECHNOLOGY
软件工程系

2、微软VSTS

- **VSTS：Visual Studio Team System，**整合了软件设计、开发、测试、部署和人员协作工具。
- **VSTS**的开发版提供了静态分析、代码剖析、代码涵盖以及其它单元测试所需的功能特性
- 进行单元测试的步骤：
  - 创建单元测试项目。
  - 设置项目引用。
  - 添加适当的测试类（一个或多个）。
  - 生成主干的单元测试框架类和属性。
  - 创建单个测试方法。
  - 创建适合特定接口的逻辑 。

VSTS常见的单元测试属性

属性	描述
TestClass()	表示一个测试装置。
TestMethod()	表示一个测试用例。
AssemblyInitialize()	在第一个 TestMethod()执行之前，进行初始化。
ClassInitialize()	在 TestClass()执行之前，进行初始化。
TestInitialize()	在执行每个 TestMethod() 之前调用，进行初始化。
TestCleanup()	在执行每个 TestMethod() 之后调用，清理测试现场。
ClassCleanup()	执行测试装置的所有 TestMethod() 之后调用，清理测试现场。
AssemblyCleanup()	最后测试现场的清理。
Description()	提供关于给定 TestMethod() 的描述。
Ignore()	由于某种原因忽略 TestMethod() 或 TestClass()。
ExpectedException()	指定的异常不是由被测代码所引发的，不影响测试结果的判断。

VSTS的单元测试断言

Assert 类	StringAssert 类	CollectionAssert 类
AreEqual()		
AreNotEqual()		AllItemsAreInstancesOfType()
AreNotSame()		AllItemsAreNotNull()
AreSame()		AllItemsAreUnique()
EqualsTests()	Contains()	AreEqual()
Fail()	DoesNotMatch()	AreEquivalent()
GetHashCodeTests()	EndsWith()	AreNotEqual()
Inconclusive()	Matches()	AreNotEquivalent()
IsFalse()	StartsWith()	Contains()
IsInstanceOfType()		DoesNotContain()
IsNotInstanceOfType()		IsNotSubsetOf()
IsNotNull()		IsSubsetOf()
IsNull()		
IsTrue()		

朱少民《软件测试》

5.1 什么是单元测试

5.2 单元测试的策略

5.3 测试用例的设计(白盒方法)

5.4 单元测试工具

5.5 集成测试

为什么总是对不上?

集成的问题

- 一些模块能顺利通过单元测试，但并不能保证它们协同工作时也能正常。
- 程序在某些局部反映不出的问题，在全局上很可能暴露出来，影响功能的发挥。
- 可能的原因有：
  - 模块相互调用时引入了新的问题；
  - 几个子功能组合起来不能实现主功能；
  - 误差不断积累达到不可接受的程度；
  - 全局变量出现访问次序问题等。

集成测试的模式

先分别测试每个模块，再把所有模块按设计要求放在一起结合成所要的程序，如大棒模式。

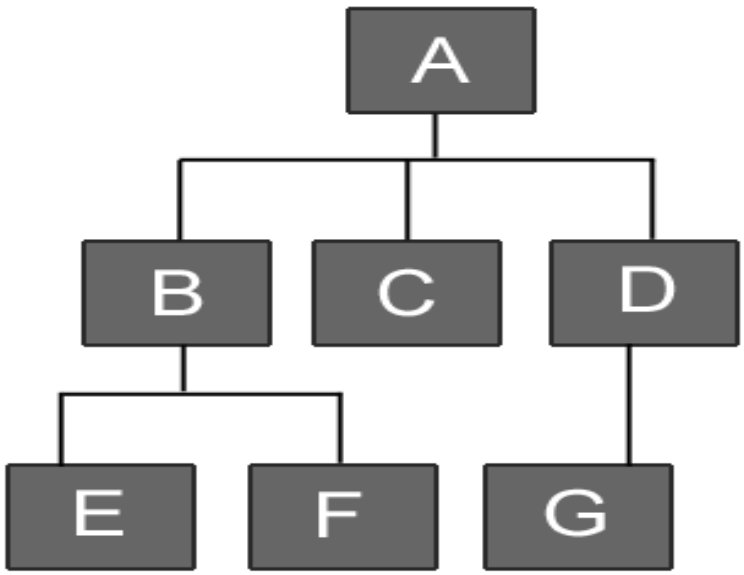
把下一个要测试的模块同已经测试好的模块结合起来进行测试，测试完以后再把下一个应该测试的模块结合进来测试。

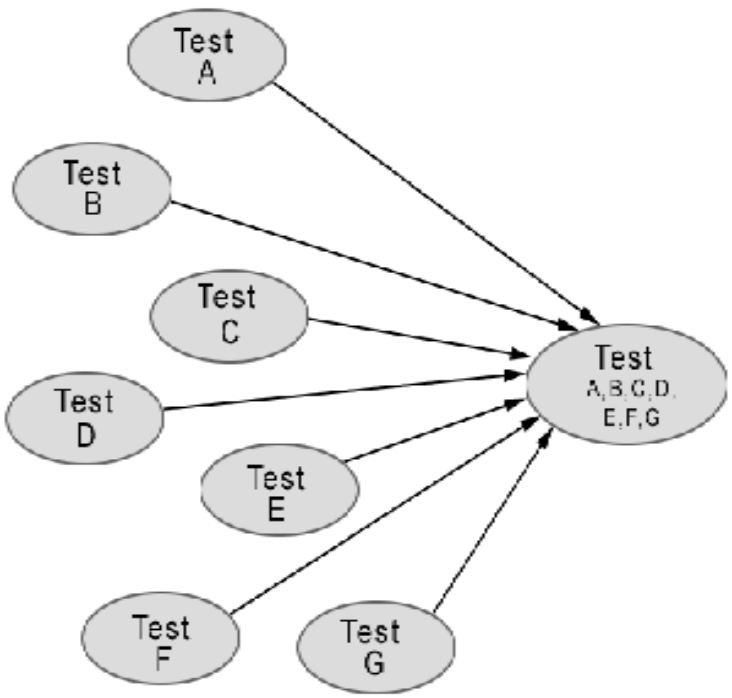
- 集成模式
  - 非增量式集成模式
  - 增量式集成模式
    - 自顶向下集成
    - 自底向上集成

从最下层的模块开始，按照程序的层次结构，逐渐形成完整的整体。

从主控模块开始，按照软件的控制层次结构，逐步把各模块集成在一起。

非增量式集成 (BigBang)

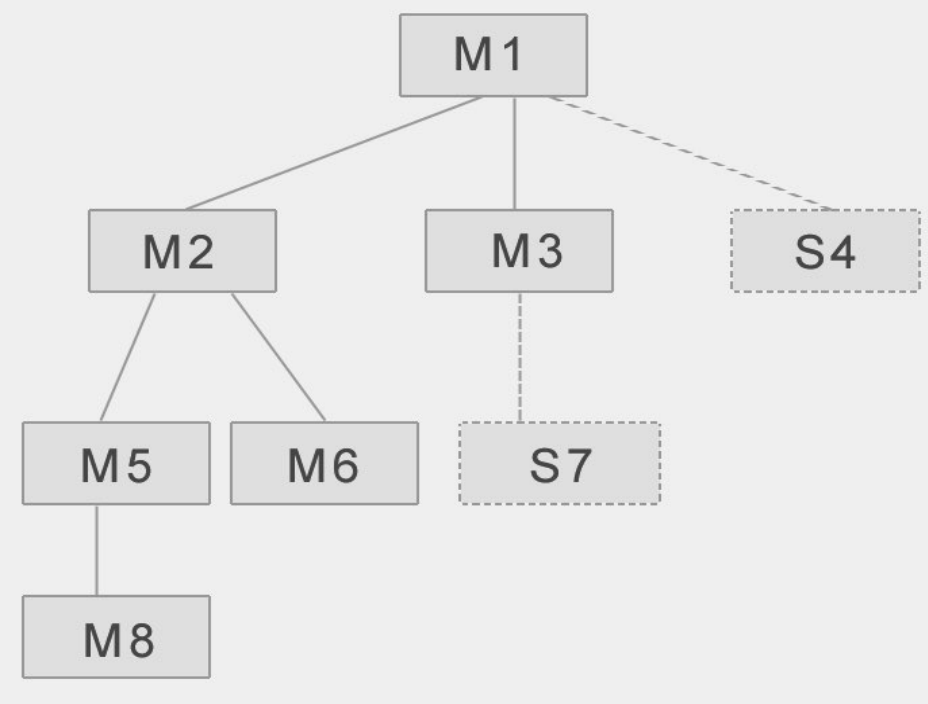




由于所有的模块一次集成的，所以很难确定出错的真正位置、所在的模块、错误的原因。

这种方法仅适用于规模较小的系统

自顶向下集成



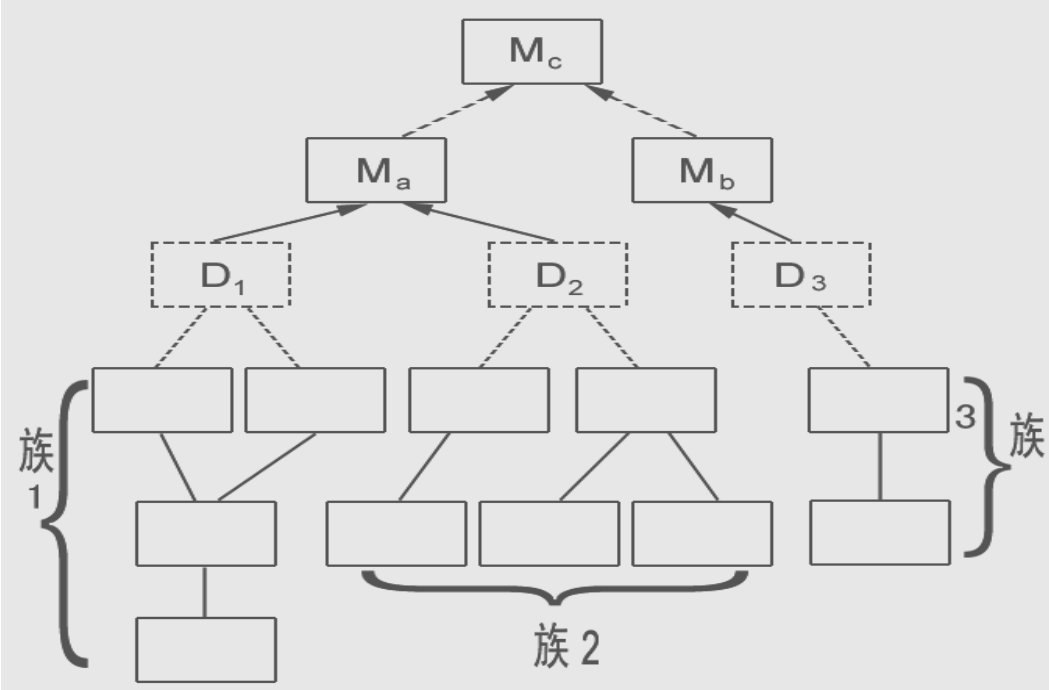
优点：模块层次越高影响面就越广，越早测试就能更早地发现问题。

缺点：需要开发大量的桩程序；且调用频繁的基础函数一般处于底层，它们的错误会发现较晚。

深度优先：**M1 – M2 – M5 – M8 – M6 – M3 – S7 – S4**

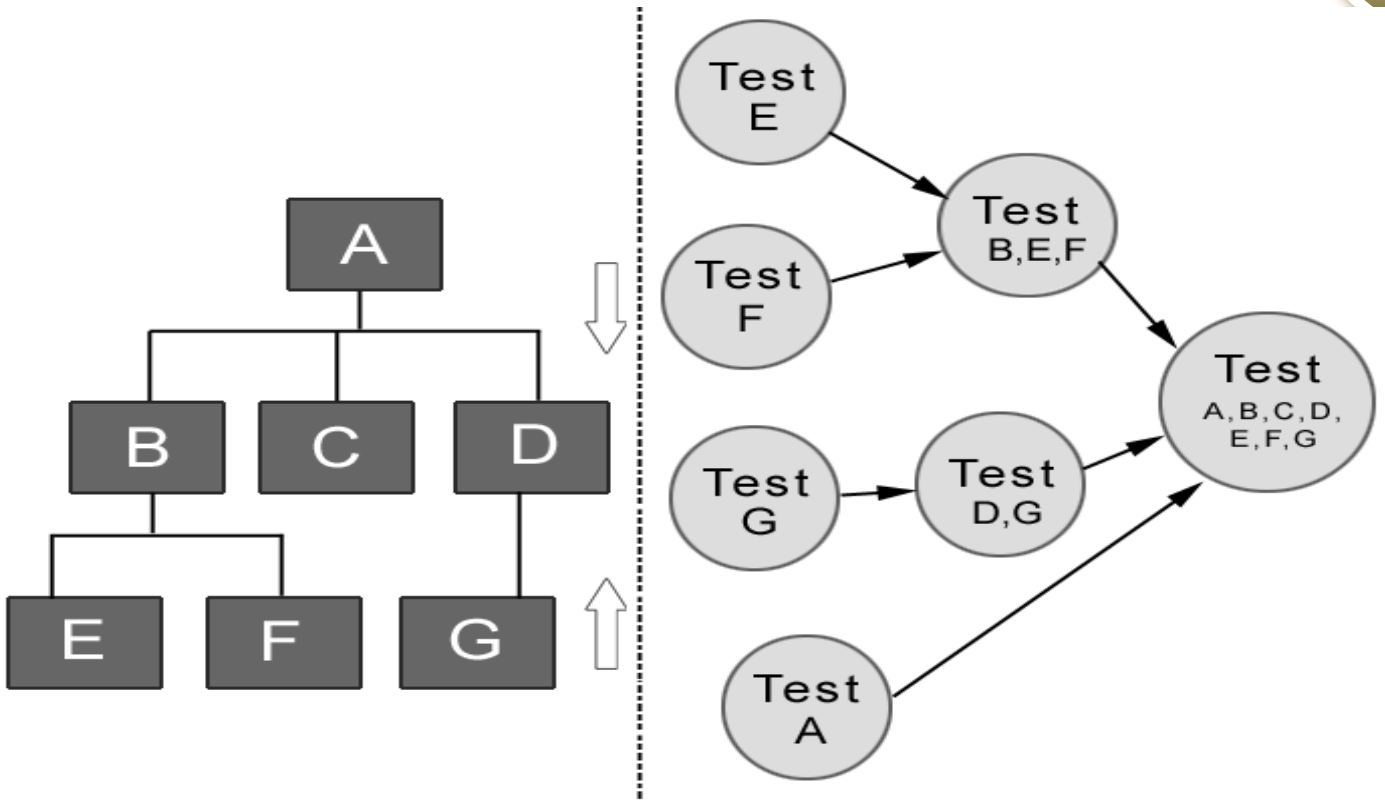
广度优先：**M1 – M2 – M3 – S4 – M5 – M6 – S7 – M8**

自底向上集成



- 优点：不需要开发桩程序，而驱动程序比较容易建立；能够较早的完成基础函数的测试，其他模块可以更早地调用这些基础函数，有利于提高开发效率，缩短开发周期。
- 缺点：影响面越广的上层模块，测试时间越靠后，后期一旦发现问题，缺陷修改越困难，风险较大。

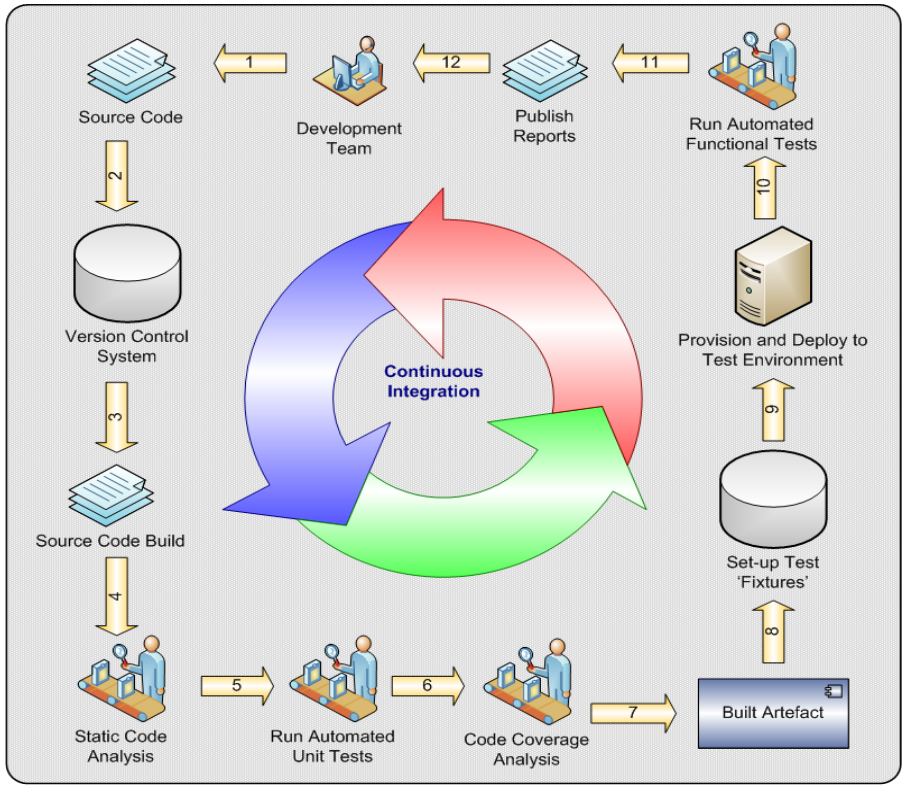
三明治集成



三明治集成法将自顶向下和自底向上有机结合，大大降低驱动程序和桩程序的编写工作量。

持续集成

持续集成是软件开发中越来越普遍的一种优秀实践，即团队开发成员经常集成他们的工作，通常每天新完成的代码至少集成一次，也就意味着每天可能会发生多次集成。



持续集成（续）

- 持续集成时，每次集成需要进行以下工作：
  - 自动的代码静态扫描
  - 自动的单元动态测试
  - 自动部署SUT：将被测系统部署到测试环境
  - 自动构建：要进行基本的功能测试，以验证构建是成功的
  - 自动生成测试报告
- 实现持续集成，需要使用相应的持续集成工具来构建一个良好的持续集成测试环境。

Q & A

The End